

RELATIONAL ER MODEL SPECIFICATION

AeroBook Flight Platform

Complete Entity-Relationship (ER) Diagram & Schema Guide

Comprehensive database engineering guide detailing all 6 isolated microservice schemas, 9 JPA tables, Crow's Foot cardinalities, and cross-service referential integrity.

DATABASE ARCHITECTURAL PARAMETERS

RDBMS Engines:

MySQL 8.0 & InnoDB Storage

Database Architecture:

Database-per-Service (6 DBs)

Visual Theme:

Black Canvas & White Marker

Notation Standard:

Crow's Foot Relational ERD

Total Persistent Tables:

9 Normalized Tables (3NF)

Integrity Strategy:

OpenFeign + Saga Compensation

Executive Summary

In cloud-native microservices architectures, data management presents a unique set of challenges and trade-offs. Monolithic architectures traditionally rely on a single shared relational database with cross-table foreign key constraints and ACID distributed transactions. However, in a resilient microservices platform, sharing databases creates high coupling, single points of failure, and scalability bottlenecks.

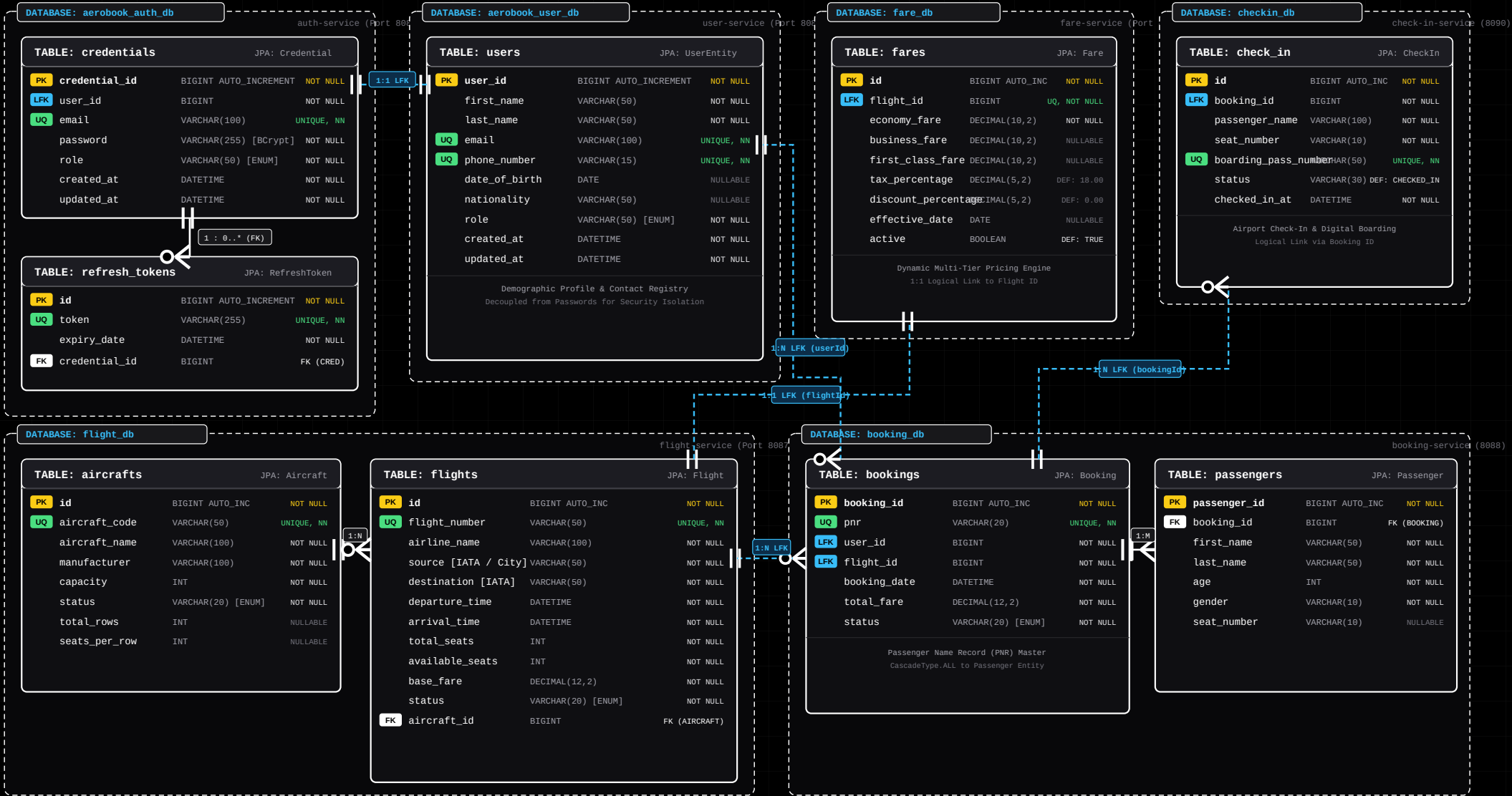
AeroBook resolves this by adopting the industry-standard Database-per-Service pattern. Each microservice (auth, user, flight, fare, booking, check-in) encapsulates its own private MySQL database schema. Physical foreign keys enforce strict integrity within each service boundary, while logical foreign keys connect entities across service boundaries through high-performance RESTful Feign contracts and eventual consistency.

DOCUMENT TABLE OF CONTENTS

Page 2	High-Resolution Entity-Relationship Diagram (Landscape Vector) Complete 6-database system ER blueprint in black canvas & white marker theme
Page 3	Section 1: Fundamentals of Relational ER Modeling in Microservices Crow's Foot notation, cardinalities (1:1, 1:N, M:N), and physical vs logical FKs
Page 4	Section 2: Identity & Traveler Relational Schemas aerobook_auth_db (credentials, refresh_tokens) and aerobook_user_db (users)
Page 5	Section 3: Flight Scheduling & Yield Fare Pricing Schemas flight_db (aircrafts, flights) and fare_db (fares multi-tier pricing)
Page 6	Section 4: Booking Orchestration & Check-In Schemas booking_db (bookings, passengers) and checkin_db (check_in boarding passes)
Page 7	Section 5: Normalization Analysis (3NF) & Distributed Data Integrity 1NF/2NF/3NF proofs, Saga compensation patterns, and no distributed 2PC locking
Page 8	Section 6: Complete Table Traceability Matrix & Technical Defense FAQ Comprehensive data dictionary and 4 critical technical review interview questions

AEROBOOK PLATFORM - SYSTEM ENTITY-RELATIONSHIP (ER) DIAGRAM

Microservices Database-per-Service Architecture • Relational Schemas & Crow's Foot Cardinality • Black Canvas & White Marker Theme



ER DIAGRAM NOTATION LEGEND & MICROSERVICES RELATIONAL INTEGRITY ARCHITECTURE

AeroBook Engineering Standard • ISO/IEC 9075 Relational Specification

CROW'S FOOT CARDINALITY NOTATION:

- One and Only One (1:1 Exactly)
- Zero or Many (0..* Optional List)
- One or Many (1..* Required At Least 1)
- Zero or One (0..1 Optional 1:1)

ATTRIBUTE BADGE TAXONOMY:

- PK** Primary Key: Unique identifier, indexed B-Tree, clustered index.
- FK** Physical Foreign Key: Intra-DB constraint, referential integrity.
- LFK** Logical Foreign Key: Inter-microservice reference across databases.
- UQ** Unique Constraint: No duplicate entries permitted (email, pnr, etc.).

DATABASE-PER-SERVICE ARCHITECTURAL RULES:

- Solid White Line:** Intra-DB physical relationship enforced by MySQL InnoDB.
- Dashed Cyan Line:** Logical cross-service link maintained at API layer via Feign.
- No distributed 2-Phase-Commit (2PC) transactions across service schemas.
- Relational schemas normalized to 3NF; zero unmanaged data anomalies.
- Each database independently scalable, backup-capable, and failover-isolated.

1. Fundamentals: Relational Modeling in Microservices

Crow's Foot Notation, Keys, and Boundary Isolation Explained in Plain English

An Entity-Relationship (ER) diagram visualizes the logical structure of a database system by declaring entities (tables), attributes (columns), primary keys (unique records), foreign keys (relationships), and cardinalities. In AeroBook, the ER diagram models the physical schemas backing the Spring Data JPA layer across all microservices.

One and Only One (||) : Strict 1:1 Association

Exactly one instance of the parent table is associated with the child. For example, every row in refresh_tokens MUST link back to exactly one credentials parent.

Zero or Many (o<) : Optional 0..* Collection

The parent entity can exist without any children, or with multiple children. For example, an aircraft can exist in fleet inventory before being assigned to any scheduled flights.

One or Many (|<-) : Mandatory 1..* Collection

The parent entity requires at least one child to be valid. In AeroBook, every Booking MUST contain at least 1 Passenger.

Zero or One (o|) : Optional 1:1 Association

A child record may or may not exist for a given parent. For example, a flight booking may exist before any check-in record has been generated.

Physical Foreign Keys vs Logical Microservice References

Solving Distributed Data Integrity Without Database Coupling

Physical Foreign Keys (Solid Lines) — Intra-Database Integrity

Physical FKs (e.g. refresh_tokens.credential_id -> credentials.credential_id, flights.aircraft_id -> aircrafts.id, passengers.booking_id -> bookings.booking_id) exist within the SAME physical MySQL schema. They are enforced directly by the MySQL InnoDB storage engine with cascading rules (ON DELETE CASCADE) and index optimization.

Logical Foreign Keys (Dashed Lines) — Cross-Database Decoupling

Cross-service references (e.g. bookings.flight_id -> flights.id, bookings.user_id -> users.user_id, fares.flight_id -> flights.id) link entities across DIFFERENT physical schemas. In microservices, physical database foreign keys cannot cross database boundaries. Instead, AeroBook enforces relational consistency at the application layer via Spring Cloud OpenFeign clients with circuit breakers.

Database-Per-Service Isolation Benefit

If flight-service experiences an unexpected surge in queries during a flash sale, its database CPU and connection pool do NOT affect the booking-service or auth-service databases. Each database can be scaled, migrated, or backed up independently.

2. Identity & Traveler Relational Schemas

aerobook_auth_db (Port 8082) and aerobook_user_db (Port 8084)

TABLE: credentials | Database: aerobook_auth_db | JPA: Credential

Primary Key: credential_id (BIGINT AUTO_INCREMENT) | Unique: email (VARCHAR 100)

- * Purpose: Dedicated table managing authentication security. Completely isolated from demographic profile data.
- * Column Breakdown:
 - credential_id : BIGINT [PK] -- Auto-increment surrogate primary key.
 - user_id : BIGINT [LFK] -- Logical reference pointing to users.user_id in user-service.
 - email : VARCHAR(100) [UQ, NOT NULL] -- Unique username used during login authentication.
 - password : VARCHAR(255) [NOT NULL] -- Irreversible BCrypt cryptographic password hash (60 chars, 12 rounds of salt).
 - role : VARCHAR(50) [NOT NULL] -- Security role enum string (ROLE_USER, ROLE_ADMIN).
 - created_at / updated_at : DATETIME [NOT NULL] -- Audit lifecycle timestamps.

TABLE: refresh_tokens | Database: aerobook_auth_db | JPA: RefreshToken

Physical FK: credential_id -> credentials.credential_id (ON DELETE CASCADE)

- * Purpose: Persists cryptographic refresh session tokens for rotating mobile and web JWT authorizations.
- * Column Breakdown:
 - id : BIGINT [PK] -- Auto-increment primary key.
 - token : VARCHAR(255) [UQ, NOT NULL] -- High-entropy UUID v4 token string.
 - expiry_date : DATETIME [NOT NULL] -- Expiration instant calculated when token is issued.
 - credential_id : BIGINT [FK, NOT NULL] -- InnoDB foreign key referencing credentials.credential_id.

TABLE: users | Database: aerobook_user_db | JPA: UserEntity

Primary Key: user_id (BIGINT AUTO_INCREMENT) | Unique: email, phone_number

- * Purpose: Houses passenger profile demographics and sensitive contact coordinates, safely decoupled from login credentials.
- * Column Breakdown:
 - user_id : BIGINT [PK] -- Auto-increment surrogate primary key.
 - first_name / last_name : VARCHAR(50) [NOT NULL] -- Legal travel names matching government passport.
 - email : VARCHAR(100) [UQ, NOT NULL] -- Customer electronic mail contact address.
 - phone_number : VARCHAR(15) [UQ, NOT NULL] -- Mobile telephone number for automated SMS flight dispatch alerts.
 - date_of_birth : DATE [NULLABLE] -- Passenger birth date for identity and infant/child ticket classification.
 - nationality : VARCHAR(50) [NULLABLE] -- Country of citizenship for international travel validation.
 - role : VARCHAR(50) [NOT NULL] -- Platform authority role, defaults to ROLE_USER.
 - created_at / updated_at : DATETIME [NOT NULL] -- Profile lifecycle audit timestamps.

SECURITY DESIGN RATIONALE: Separation of Credentials from Profiles

By storing passwords in aerobook_auth_db and demographics in aerobook_user_db, AeroBook prevents single-vector data breaches. Customer support representatives querying user demographics have zero access to BCrypt password hashes or session tokens. Furthermore, GDPR "Right to be Forgotten" anonymization can be executed on aerobook_user_db without invalidating audit logs.

3. Flight Scheduling & Yield Fare Pricing Schemas

flight_db (Port 8087) and fare_db (Port 8089)

TABLE: aircrafts | Database: flight_db | JPA: Aircraft

Primary Key: id (BIGINT AUTO_INCREMENT) | **Unique:** aircraft_code (VARCHAR 50)

- * Purpose: Airline commercial fleet catalog defining aircraft airframes, seat layouts, and operational states.
- * Column Breakdown:
 - id : BIGINT [PK] -- Auto-increment aircraft primary key.
 - aircraft_code : VARCHAR(50) [UQ, NOT NULL] -- Unique aircraft registration tail code (e.g. A320-001).
 - aircraft_name : VARCHAR(100) [NOT NULL] -- Commercial model designation (e.g. Airbus A320neo, Boeing 737-800).
 - manufacturer : VARCHAR(100) [NOT NULL] -- Aerospace manufacturer company (Airbus, Boeing, Embraer).
 - capacity : INT [NOT NULL] -- Maximum passenger seating capacity.
 - status : VARCHAR(20) [NOT NULL] -- Operational fleet status enum (ACTIVE, MAINTENANCE, RETIRED).

TABLE: flights | Database: flight_db | JPA: Flight

Physical FK: aircraft_id -> aircrafts.id | **Logical Links:** fares.flight_id, bookings.flight_id

- * Purpose: Master timetable of all commercial airline flights, real-time seat inventory, and operational statuses.
- * Column Breakdown:
 - id : BIGINT [PK] -- Auto-increment scheduled flight primary key.
 - flight_number : VARCHAR(50) [UQ, NOT NULL] -- Commercial flight code (e.g. AI101, 6E204).
 - airline_name : VARCHAR(100) [NOT NULL] -- Operating carrier brand (e.g. Air India, IndiGo).
 - source / destination : VARCHAR(50) [NOT NULL] -- Origin and arrival IATA airport codes (e.g. DEL, BOM).
 - departure_time / arrival_time : DATETIME [NOT NULL] -- Scheduled takeoff and touchdown instants.
 - total_seats / available_seats : INT [NOT NULL] -- Total capacity and live remaining bookable seat inventory.
 - base_fare : DECIMAL(12,2) [NOT NULL] -- Baseline economy ticket fare before class multipliers.
 - status : VARCHAR(20) [NOT NULL] -- SCHEDULED, DELAYED, CANCELLED, COMPLETED.
 - aircraft_id : BIGINT [FK, NOT NULL] -- Physical FK referencing aircrafts.id.

TABLE: fares | Database: fare_db | JPA: Fare

Primary Key: id (BIGINT AUTO_INCREMENT) | **Logical 1:1 Link:** flight_id (BIGINT UNIQUE)

- * Purpose: Yield management and dynamic cabin class pricing engine decoupled from the core flight timetable.
- * Column Breakdown:
 - id : BIGINT [PK] -- Auto-increment pricing rule primary key.
 - flight_id : BIGINT [LFK, UQ, NOT NULL] -- Logical reference pointing to flights.id in flight_db.
 - economy_fare : DECIMAL(10,2) [NOT NULL] -- Base ticket price for standard Economy Class seating.
 - business_fare : DECIMAL(10,2) [NULLABLE] -- Premium price for Business Class luxury seating.
 - first_class_fare : DECIMAL(10,2) [NULLABLE] -- Ultra-premium price for First Class seating.
 - tax_percentage : DECIMAL(5,2) [DEFAULT 18.00] -- Statutory airport and government tax rate (18% GST).
 - discount_percentage : DECIMAL(5,2) [DEFAULT 0.00] -- Promotional coupon or corporate discount percentage.
 - effective_date : DATE [NULLABLE] & active : BOOLEAN [DEFAULT TRUE] -- Tariff activation controls.

YIELD PRICING DESIGN PATTERN: Why Separate Fare from Flight?

Separating fare pricing into fare_db allows revenue management algorithms to dynamically adjust cabin multipliers, tax rules, and flash sales without placing write locks on the critical flights timetable table in flight_db.

4. Booking Orchestration & Check-In Schemas

booking_db (Port 8088) and checkin_db (Port 8090)

TABLE: bookings | Database: booking_db | JPA: Booking

Primary Key: booking_id (BIGINT) | **Unique:** pnr (VARCHAR 20) | **Composition:** 1 *-- 1..* passengers

- * Purpose: Core reservation aggregate representing ticket sales, payment confirmation, and PNR tracking.
- * Column Breakdown:
 - booking_id : BIGINT [PK] -- Auto-increment primary key.
 - pnr : VARCHAR(20) [UQ, NOT NULL] -- Alphanumeric Passenger Name Record (e.g. PNR-AB789) generated at confirmation.
 - user_id : BIGINT [LFK, NOT NULL] -- Logical reference pointing to customer profile in user-service.
 - flight_id : BIGINT [LFK, NOT NULL] -- Logical reference pointing to scheduled flight in flight-service.
 - booking_date : DATETIME [NOT NULL] -- Timestamp when ticket was purchased and confirmed.
 - total_fare : DECIMAL(12,2) [NOT NULL] -- Aggregate reservation cost (fare sum + statutory taxes - discounts).
 - status : VARCHAR(20) [NOT NULL] -- Lifecycle state enum: BOOKED, CANCELLED.

TABLE: passengers | Database: booking_db | JPA: Passenger

Physical FK: booking_id -> bookings.booking_id (CascadeType.ALL, orphanRemoval = true)

- * Purpose: Individual traveler records traveling under the parent booking reservation.
- * Column Breakdown:
 - passenger_id : BIGINT [PK] -- Auto-increment primary key.
 - booking_id : BIGINT [FK, NOT NULL] -- Physical FK referencing parent bookings.booking_id.
 - first_name / last_name : VARCHAR(50) [NOT NULL] -- Individual passenger travel identity names.
 - age : INT [NOT NULL] & gender : VARCHAR(10) [NOT NULL] -- Demographics for flight manifest.
 - seat_number : VARCHAR(10) [NULLABLE] -- Assigned cabin seat coordinate (e.g. 12A, 14B).

TABLE: check_in | Database: checkin_db | JPA: CheckIn

Primary Key: id (BIGINT) | **Unique:** boarding_pass_number (VARCHAR 50) | **Logical FK:** booking_id

- * Purpose: Airport web check-in verification, boarding gate authorization, and mobile boarding pass delivery.
- * Column Breakdown:
 - id : BIGINT [PK] -- Auto-increment primary key.
 - booking_id : BIGINT [LFK, NOT NULL] -- Logical reference pointing to bookings.booking_id in booking-service.
 - passenger_name : VARCHAR(100) [NOT NULL] -- Full traveler legal name confirmed during boarding.
 - seat_number : VARCHAR(10) [NOT NULL] -- Allocated physical cabin seat coordinate.
 - boarding_pass_number : VARCHAR(50) [UQ, NOT NULL] -- Official unique barcode reference (e.g. BP-1001-12A).
 - status : VARCHAR(30) [NOT NULL] -- Operational state, defaults to CHECKED_IN.
 - checked_in_at : DATETIME [NOT NULL] -- Exact timestamp when passenger completed web check-in.

COMPOSITION LIFECYCLE: Booking *-- Passenger

The relationship between bookings and passengers is an authentic UML/ER Composition. Passengers cannot exist independently without a parent booking. When a booking is purged or cancelled, Spring Data JPA and MySQL InnoDB automatically cascade the operation to purge all child passenger records.

5. Normalization Analysis (3NF) & Data Integrity

Mathematical Proof of Third Normal Form & Eventual Consistency

First Normal Form (1NF) Proof — Atomicity & No Repeating Groups

Every column across all 9 tables holds atomic, indivisible scalar values. For instance, customer full names are split into `first_name` and `last_name`; flight schedules separate `departure_time` and `arrival_time`. No table stores comma-separated lists or serialized JSON in relational columns. Each row is uniquely identified by a primary key.

Second Normal Form (2NF) Proof — Full Functional Dependency

A table is in 2NF if it is in 1NF and no non-prime attribute is partially dependent on any candidate key. Because every single table in AeroBook uses an artificial surrogate primary key (`BIGINT AUTO_INCREMENT`), all non-key attributes are fully functionally dependent on the entire primary key, completely eliminating partial dependencies.

Third Normal Form (3NF) Proof — Elimination of Transitive Dependencies

A table is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on another non-prime attribute. For example, `bookings` does NOT store `airline_name`, `source`, or `destination` (which are stored in `flights`). Nor does it store the customer's email or phone (which are stored in `users`). Attributes depend ONLY on the primary key, strictly satisfying 3NF.

Distributed Consistency Without Two-Phase-Commit (2PC)

How AeroBook Manages Cross-Database Transactions

Client-Side Validation via Declarative OpenFeign

When a customer requests a flight booking via `POST /api/bookings`, `booking-service` invokes `flight-service` via OpenFeign to verify flight existence and available seat count before persisting the reservation row. If seats are depleted, the transaction halts immediately with `HTTP 409 Conflict`.

Saga Compensating Actions on Reservation Cancellation

When a reservation is cancelled via `PUT /api/bookings/{id}/cancel`, `booking-service` marks the status as `CANCELLED` in `booking_db` and dispatches a compensating request to `flight-service` to increment `available_seats` back up in `flight_db`, maintaining eventual consistency without distributed table locks.

Idempotent Seat Allocation During Airport Check-In

`check-in-service` prevents double-assignment of seats through an application-level seat matrix lookup and an indexed `UNIQUE` constraint on `boarding_pass_number`, ensuring reliable concurrency control.

6. Relational Traceability Matrix & Technical Defense FAQ

Master Data Dictionary & Critical Technical Review Defense

COMPLETE SCHEMA TRACEABILITY MATRIX

credentials	aerobook_auth_db	auth-service	credential_id	LFK: user_id
refresh_tokens	aerobook_auth_db	auth-service	id	FK: credential_id
users	aerobook_user_db	user-service	user_id	Referenced by auth & booking
aircrafts	flight_db	flight-service	id	Referenced by flights
flights	flight_db	flight-service	id	FK: aircraft_id LFK: fares, bookings
fares	fare_db	fare-service	id	LFK: flight_id (UQ)
bookings	booking_db	booking-service	booking_id	LFK: user_id, flight_id
passengers	booking_db	booking-service	passenger_id	FK: booking_id (CASCADE)
check_in	checkin_db	check-in-service	id	LFK: booking_id

Technical Review & Viva Defense Questions

Direct Answers to Key Architectural Challenges

Q1: Why didn't you put all tables into a single monolithic MySQL database?

A single shared database creates tight coupling between microservices. If multiple teams write to the same database, schema migrations require coordinated platform downtime. Furthermore, a failure or heavy query in flight-service would exhaust database connections for critical user login and booking transactions. Database-per-service provides total fault isolation and independent horizontal scalability.

Q2: How do you enforce foreign key constraints when tables are in separate databases?

Microservices replace physical database foreign keys with application-level contracts. We use Spring Cloud OpenFeign to validate reference existence (e.g. checking flightId before booking creation). In addition, Resilience4j Circuit Breakers provide graceful fallback handling, and asynchronous compensating transactions ensure eventual consistency.

Q3: Why is the relationship between Booking and Passenger modeled as a Composition?

A passenger ticket cannot exist without a valid booking reservation. In JPA, this is mapped using CascadeType.ALL and orphanRemoval = true. When a booking reservation is deleted or purged, Hibernate and MySQL InnoDB automatically clean up all associated passenger records, ensuring zero orphaned child data.

Q4: How does the system handle concurrent booking attempts for the final available seat?

flight-service protects seat inventory by executing seat decrements using database row-level locking (SELECT ... FOR UPDATE or atomic UPDATE flights SET available_seats = available_seats - ? WHERE id = ? AND available_seats >= ?). This guarantees ACID serialized isolation for concurrent requests without phantom reads.

